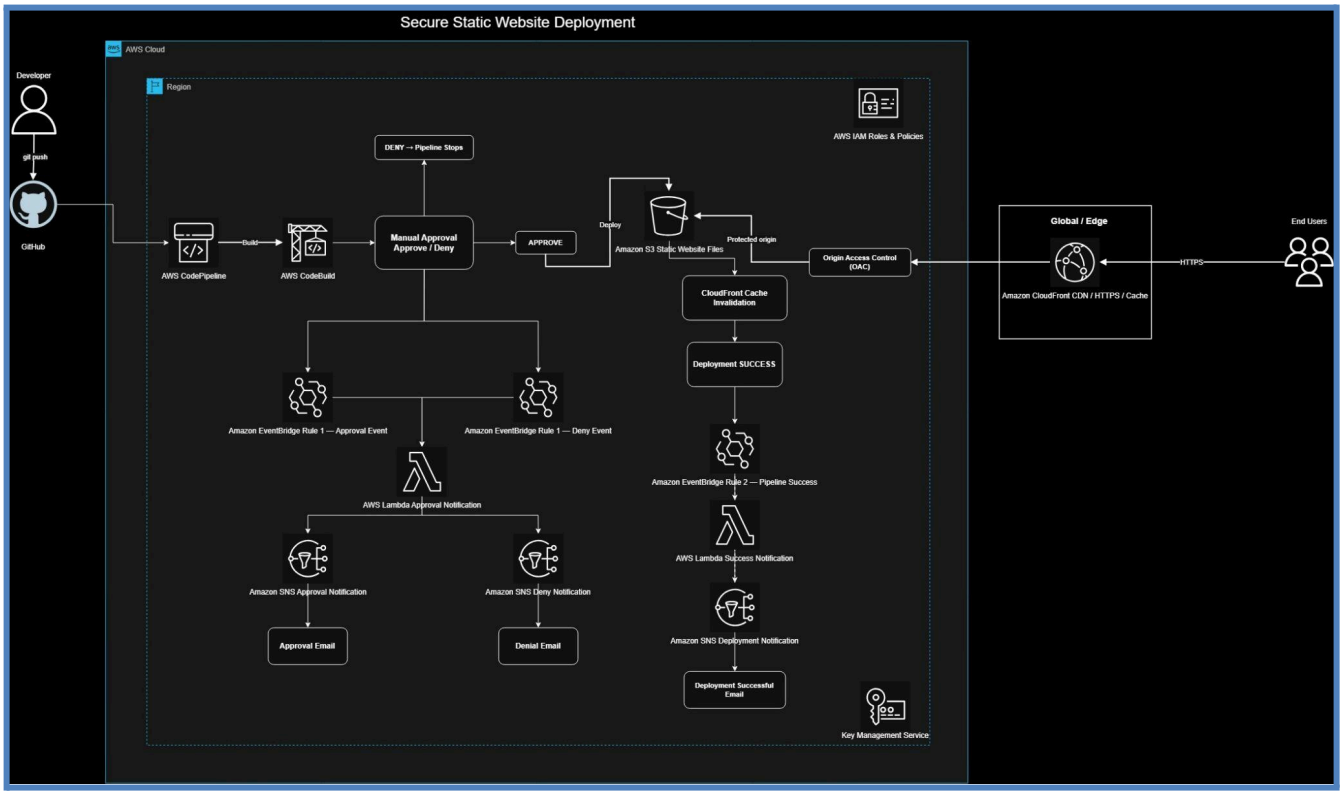
PROJECT REPORT

Static Website Deployment with Secure CI/CD

AWS Infrastructure as Code • Secure CI/CD • CloudFront Delivery

Project Details	Value
Project	Static Website Deployment with Secure CI/CD
Cloud Platform	Amazon Web Services (AWS)
Infrastructure as Code	AWS CloudFormation
Source Repository	GitHub
CI/CD	AWS CodePipeline + AWS CodeBuild
Content Storage	Amazon S3
Content Delivery	Amazon CloudFront
Notifications	Amazon EventBridge + AWS Lambda + Amazon SNS
Encryption	AWS KMS



1. Executive Summary

This project implements a secure and automated deployment platform for a static website on Amazon Web Services (AWS). GitHub is used as the source repository, AWS CodePipeline and AWS CodeBuild provide CI/CD automation, Amazon S3 stores the website as a private origin, and Amazon CloudFront provides secure HTTPS content delivery.

Production deployment is protected by a manual approval stage. Amazon EventBridge captures pipeline events, an AWS Lambda function formats deployment notifications, and an encrypted Amazon SNS topic delivers them by email. After deployment, CloudFront cache invalidation ensures users receive the latest website content.

The complete infrastructure is defined using AWS CloudFormation, making the solution repeatable, versionable, and auditable. The final implementation was validated through successful pipeline executions, notification testing, CloudFront cache invalidation, and CloudFormation drift detection.

2. Objective

Design and implement a secure, automated deployment solution for a static website on AWS using Infrastructure as Code, controlled production approval, private S3 hosting, CloudFront delivery, and automated deployment notifications.

- Deploy a static website on AWS.
- Serve the website through Amazon CloudFront.
- Keep the Amazon S3 website bucket private and prevent public access.
- Provision infrastructure using AWS CloudFormation.
- Use GitHub as the source repository.
- Implement CI/CD using AWS CodePipeline and AWS CodeBuild.
- Automatically trigger the pipeline for changes to the main branch.
- Pause production deployment for manual approval via email notification.
- Deploy the approved website version to S3 and make it available through CloudFront.
- Invalidate CloudFront cache after successful deployment.
- Follow security best practices and least privilege.

3. Project Requirements and Implementation

Requirement	Implementation
Static website	Website files are stored in Amazon S3 and deployed through CodePipeline.
CloudFront	CloudFront provides HTTPS delivery to end users.
Private S3	S3 Block Public Access remains enabled; CloudFront accesses the origin through OAC.
IaC	AWS CloudFormation defines the infrastructure.
Source control	GitHub stores the source code.
CI/CD	CodePipeline orchestrates the workflow and CodeBuild performs validation/build.
Main branch trigger	Changes to the main branch initiate the pipeline.
Manual production approval	Pipeline pauses before production deployment.
Approval notifications	EventBridge + Lambda + SNS send approval and decision emails.
Latest content	CloudFront cache invalidation runs after deployment.
Security	IAM least privilege, private S3, OAC, KMS, HTTPS, and manual approval are used.

4. Solution Architecture

The final architecture separates source control, build, approval, deployment, content delivery, and notification responsibilities across appropriate services.

4.1 Deployment Flow

1. The developer pushes changes to the main branch in GitHub.
2. CodePipeline detects the source change and starts a pipeline execution.
3. CodeBuild validates the frontend and prepares the deployment artifact.
4. The pipeline pauses at the Manual Approval stage.
5. After approval, the website artifact is deployed to the private S3 website bucket.
6. The pipeline performs CloudFront cache invalidation.
7. CloudFront serves the updated website to end users over HTTPS.

4.2 Notification Flow

Event	Rule / Service	Notification
Approval request / IN_PROGRESS	EventBridge Approval Status rule	Deployment approval email with pipeline information.
Approval SUCCEEDED	EventBridge Approval Status rule	Deployment approved email.
Approval FAILED / denied	EventBridge Approval Status rule	Deployment denied email.
CacheInvalidation SUCCEEDED	EventBridge Deployment Success rule	Deployment successful email with CloudFront URL.

The notification architecture uses two EventBridge rules, one notification Lambda function, and one encrypted SNS deployment-status topic.

4.3 Security Architecture

- Private S3 origin with Block Public Access enabled.
- CloudFront Origin Access Control (OAC) for protected S3 access.
- Dedicated IAM service roles with required permissions.
- Customer-managed KMS key for protected encryption operations.
- Encrypted SNS deployment-status topic.
- Manual approval before production deployment.

5. AWS Services Used

Service	Purpose
AWS CloudFormation	Infrastructure as Code.
Amazon S3	Private website storage and CodePipeline artifact storage.
Amazon CloudFront	Global HTTPS content delivery.
CloudFront OAC	Secure authorization from CloudFront to the private S3 origin.
AWS CodePipeline	CI/CD orchestration.
AWS CodeBuild	Frontend validation and artifact generation.
Amazon EventBridge	Pipeline event detection for approval and successful cache invalidation.
AWS Lambda	Formats and publishes deployment notifications.
Amazon SNS	Email notification delivery.
AWS KMS	Customer-managed encryption key management.
AWS IAM	Access control and least-privilege permissions.
Amazon CloudWatch Logs	Pipeline, build, and Lambda logging.
GitHub	Source-code repository.

6. Infrastructure as Code

AWS CloudFormation defines the project infrastructure in a repeatable and auditable template. The stack is named SecureStaticWebsiteDeployment.

- Private website S3 bucket.
- CodePipeline artifact bucket.
- CloudFront distribution and Origin Access Control.
- CodePipeline and service role.
- CodeBuild project and service role.
- Notification Lambda and execution role.
- SNS deployment-status topic and subscription configuration.
- Two EventBridge rules and Lambda invoke permissions.
- Customer-managed KMS key.
- CloudWatch logging resources and permissions.

6.1 CloudFormation Parameters

Parameter	Purpose
GitHubConnectionArn	Existing GitHub CodeConnections connection ARN.
NotificationEmail	Email address used for deployment notifications.
Owner	Project owner value/tag.
ProjectName	Project name used in resource naming and tagging.
Environment	Environment identifier; Production.
DevelopedBy	Developer/project attribution.
GitHubBranch	Pipeline source branch; main.
GitHubRepository	Repository in owner/repository format.

7. CI/CD Pipeline

Stage / Action	Purpose
Source	Retrieves the latest source from GitHub main.
Build	CodeBuild validates the frontend and creates the deployment artifact.
Manual Approval	Pauses production deployment until approval or denial.
DeployWebsite	Deploys the approved artifact to the private S3 bucket.
CacheInvalidation	Invalidates CloudFront cache after deployment.

7.1 Build Validation

The build validates the expected frontend structure:

- Frontend/index.html
- Frontend/style.css
- Frontend/script.js

The Frontend directory is published as the deployment artifact.

8. Production Approval and Notification System

When CodePipeline reaches the manual approval action, the approval-status event is processed through EventBridge and the notification Lambda. Lambda generates the appropriate message and publishes it to the encrypted SNS topic.

The same workflow handles approval decisions. An approved decision produces a Deployment Approved notification, while a denied decision produces a Deployment Denied notification.

After an approved deployment reaches the CloudFront cache invalidation action, a successful invalidation event triggers the second EventBridge rule. The Lambda then sends the final Deployment Successful email containing the CloudFront website URL.

9. Security Implementation

9.1 S3 and CloudFront

- S3 Block Public Access is enabled.
- The website bucket does not allow public object access.
- CloudFront is the public delivery endpoint.
- Origin Access Control protects the S3 origin.

9.2 IAM

Separate IAM roles are used for CodePipeline, CodeBuild, and Lambda. Permissions are scoped to the operations required by each service. The notification Lambda role includes logging, SNS publishing, required KMS access, and the CodePipeline read permissions used by the notification workflow.

9.3 KMS and SNS

A customer-managed KMS key is included in the solution. The notification Lambda role has kms:Decrypt and kms:GenerateDataKey permissions required for encrypted SNS publishing.

9.4 HTTPS

End users access the website through the CloudFront distribution over HTTPS, while the S3 origin remains private.

10. Troubleshooting and Resolution

Issue	Resolution
S3/CloudFront access denied	Configured CloudFront Origin Access Control and the required S3 bucket policy.
KMS permissions	Configured KMS key policy and Lambda execution permissions required for encrypted SNS publishing.
CodePipeline logging failure	Added logs:CreateLogGroup, logs:CreateLogStream, and logs:PutLogEvents permissions to the pipeline service role.
Approval notification not initially triggered	Corrected the EventBridge approval-status event pattern and notification workflow.
Duplicate Lambda SNS/KMS policy caused drift	Removed the manually added duplicate policy so the role matched the CloudFormation template.
CloudFormation drift detected	Investigated NotificationLambdaRole, removed the extra policy, and reran drift detection.
Local CloudFormation template text-decoding issue	The template was uploaded directly through the CloudFormation stack workflow when local CLI decoding prevented validation.

11. Testing and Validation

- GitHub push successfully triggers CodePipeline.
- CodeBuild completes successfully.
- Manual approval correctly pauses production deployment.
- Approval notification email is received.
- Approval decision notifications are received for approved and denied states.
- Approved deployment proceeds to S3.
- CloudFront cache invalidation completes successfully.
- Deployment-success email is received with the CloudFront URL.
- Updated website content is visible through CloudFront after invalidation.
- EventBridge rules show successful matching/invocation during successful tests.
- CloudFormation drift detection returns IN_SYNC for the stack and supported resources.

12. Final AWS Resource Inventory

Resource	Final State
CloudFormation stack	SecureStaticWebsiteDeployment — IN_SYNC
S3	Private website bucket + CodePipeline artifact bucket; CloudFormation template staging buckets retained.
CloudFront	One enabled distribution serving the private S3 origin.
CodePipeline	One production deployment pipeline.
CodeBuild	One build project.
Lambda	One notification Lambda function.
SNS	One deployment-status topic with confirmed email subscription.
EventBridge	Two project rules: ApprovalStatus and DeploymentSuccess.
KMS	One customer-managed project key.
CloudWatch Logs	Required CodeBuild, Lambda, and CodePipeline log groups.

13. Deliverables

Deliverable	Implementation
Source code repository	GitHub repository containing the static website source.
Infrastructure as Code	AWS CloudFormation template.
CI/CD configuration	AWS CodePipeline and CodeBuild configuration.
Architecture & Flow diagram	Final AWS architecture diagram
README	Repository documentation explaining architecture, deployment, and usage.
Final project report	This technical report.

14. Repository Structure

```
Secure-Static-Website-Deployment/
├── Frontend/
│   ├── index.html
│   ├── style.css
│   └── script.js
├── cloudformation/
│   └── main.yml
├── buildspec.yml
└── README.md
```

15. Project Outcomes

- Static website securely hosted with a private S3 origin.
- Global HTTPS delivery through CloudFront.
- Automated GitHub-to-production CI/CD workflow.
- Manual production approval gate.
- Automated approval, denial, and successful-deployment email notifications.
- CloudFront cache invalidation after deployment.
- Repeatable AWS infrastructure through CloudFormation.
- IAM, KMS, S3, CloudFront, EventBridge, Lambda, and SNS integrated into the security and automation design.
- Final infrastructure validated with CloudFormation drift status IN_SYNC.

16. Conclusion

The Static Website Deployment with Secure CI/CD project provides a complete AWS-based deployment solution combining Infrastructure as Code, continuous integration and delivery, secure content storage, global HTTPS distribution, controlled production approval, and event-driven notifications.

The architecture separates source control, build, approval, deployment, content delivery, and notification responsibilities across appropriate services. The S3 origin remains private while CloudFront provides the public access layer. IAM and KMS provide security controls, while EventBridge, Lambda, and SNS provide automated deployment visibility.

The final implementation was validated end-to-end, including deployment, approval, cache invalidation, notification, and infrastructure drift validation. The resulting architecture is automated, secure, repeatable, and aligned with the project requirements and AWS best practices.

Appendix A — Key Deployment Flow

GitHub → CodePipeline → CodeBuild → Manual Approval → S3 Deployment → CloudFront Cache Invalidation → CloudFront

Appendix B — Notification Flow

CodePipeline Event → EventBridge → Notification Lambda → Encrypted SNS Topic → Email

End of Report